Reproducing and Extending #GNN: Hashing-Accelerated Graph Neural Networks for Link Prediction

V Sai Siddhartha
Roll No: 22B0954

V Gnana Mahesh
Roll No: 22B0949

V Madhav
Roll No: 22B0993

May 9, 2026

1 Introduction: Link Prediction

A *network* (or graph) is a set of nodes connected by edges. In an *attributed* network, each node also carries a feature vector — for example, a Twitter user is a node, follow-relationships are edges, and the user's posted hashtags form their feature vector.

Link prediction asks: given the network as observed today, which currently-unconnected pairs of nodes are most likely to form an edge in the future, or which observed edges have we failed to record? The task is fundamental in social-network friend recommendation, e-commerce product recommendation, biological protein-interaction prediction, and knowledge-graph completion. Formally, we hide a fraction of the edges, run a model on the remaining graph, and test whether the hidden (positive) edges score higher than randomly-sampled non-edges. The standard ranking metric is the *Area Under the ROC Curve* (AUC): the probability that a randomly chosen positive pair is ranked above a randomly chosen negative one. AUC of 1.0 is perfect; 0.5 is random.

2 Why Hashing is Better than Standard GNNs Here

A standard graph neural network (GNN) such as GCN, GraphSAGE, or GAT maintains a learned embedding for every node. Embeddings are randomly initialized and gradient-trained so that connected nodes end up close in vector space. This works well, but it has practical costs:

- **Training time.** Backpropagation through several layers on a large graph is expensive; the paper reports that SEAL and P-GNN take hours on the larger datasets.
- **Hardware.** Learned GNNs typically need a GPU.
- **Hyperparameter sensitivity.** Learning rates, batch sizes, regularizers, and architectural choices all need tuning.
- **Inductive limits.** A new node added to the graph after training requires either fine-tuning or a separate inductive treatment.

#GNN replaces the learned aggregation step with a fixed, parameter-free random-projection scheme based on MinHash. The practical consequences are:

- **No training, no gradients.** Embeddings are produced in a single forward pass. The paper reports speedups of two to four orders of magnitude over learned GNNs.
- **CPU-only.** The whole algorithm reduces to integer arithmetic and `argmin` operations.
- **Theoretically grounded similarity.** MinHash collision probability equals the Jaccard similarity of the underlying sets, so the Hamming similarity of two embeddings is an unbiased estimator of a meaningful neighborhood-similarity metric.
- **Inductive by construction.** A new node is hashed using the same fixed family; no retraining required.

The trade-off is a small AUC gap on some datasets compared to the best learned GNNs. #GNN's contribution is showing that this gap is surprisingly small while the speed gain is enormous.

3 The #GNN Algorithm

We summarize Algorithm 1 of the paper.

Input. An attributed graph $G = (V, E, X)$ where each node $v \in V$ has feature set $X_v \subseteq \mathcal{A}$ (a subset of the attribute vocabulary), the number of iterations T, and the embedding dimension K.

Hash family. For each iteration t and each dimension $k \in \{1, \ldots, K\}$, the algorithm samples three independent permutation-style hash functions of the universal form $\pi(i) = (a\,i + b) \bmod c$, where c is a prime $\geq |\mathcal{A}|$ and a, b are sampled uniformly at random. The three functions are denoted π_1 (self), π_2 (neighbor), and π_3 (diffusion).

Two-phase update per iteration. Let $x_v^{(t-1)} \subseteq \mathcal{A}$ be node v's representation before iteration t and $\mathcal{N}(v)$ its neighborhood.

Phase 1 — Diffusion. For every node v, compute a diffused message:

$$m_v^{(t,k)} \;=\; \arg\min_{a \in x_v^{(t-1)}} \pi_3(a).$$

Phase 2 — Aggregation. For every node v, compute the k-th coordinate of its new representation as

$$x_v^{(t)}[k] \;=\; \arg\min \Big\{ \pi_1(a) : a \in x_v^{(t-1)} \Big\} \cup \Big\{ \pi_2(m_u^{(t,k)}) : u \in \mathcal{N}(v) \Big\}.$$

That is, the algorithm takes the element with the smallest hash value across the union of v's own hashed features and its neighbors' hashed diffused messages.

After T iterations, the embedding of v is the length-K vector $x_v^{(T)}$. The similarity of two nodes is the *Hamming similarity* $\mathrm{Sim}(u, v) = \frac{1}{K} \sum_k \mathbf{1}[x_u^{(T)}[k] = x_v^{(T)}[k]]$. Crucially, the expected value of this Hamming similarity equals the Jaccard similarity of the underlying T-hop neighborhoods (paper §3.3), which is what makes the method work without learning.

Complexity. Time is $\mathcal{O}(T \cdot K \cdot |V| \cdot (\text{ avg degree } + |X_v|))$, linear in the number of edges. No training, no GPU.

4 Modifications

We propose three modifications, each independently toggleable, and an "all-three" combination that activates them simultaneously.

A. Jumping-Knowledge Concatenation (`-jk concat`)

The paper outputs only the embedding from the final iteration T, discarding intermediate iterations $1, \ldots, T-1$. But different node pairs benefit from different neighborhood depths: a short-range pair is best characterized by iteration 1 (one-hop overlap), while a long-range pair needs iteration T. Inspired by Jumping-Knowledge networks [2], we concatenate embeddings from *all* iterations into a single $T{\cdot}K$ length vector, exposing all proximity scales to the similarity computation.

Cost. The Hamming-similarity comparison becomes T times more expensive in bits, but K is small (200) so the total is still cheap.

B. Self-Influence Parameter α (`-alpha 0.7`)

In the paper's Phase 2, self-features and neighbor messages are merged into one set before `argmin`. The relative contribution of *self* versus *neighbors* is therefore implicit and depends on $|X_v|$ versus $|\mathcal{N}(v)|$. In dense graphs this means neighbors dominate; in sparse ones, self does. We make the balance explicit: with probability α the per-dimension `argmin` is taken over self-features only, otherwise over neighbor messages only.

C. Random Neighborhood Cap (`-neigh-cap 30`)

High-degree hubs in social networks contribute very large neighbor sets to Phase 2. The intuition is that randomly subsampling each neighborhood to at most 30 nodes might denoise hub contributions and improve AUC.

5 Experimental Setup

Datasets. We run on three of the five attributed networks shipped with the official repository: `twitter`, `facebook`, and `blog`. We use the *full* datasets (no subsetting) since this project focuses only on the link-prediction study, not the scalability or parameter sensitivity studies of the paper. Table 1 summarizes the size and density of each dataset.

Table 1: Dataset statistics. "Avg. train degree" is the average node degree in the *training graph* (80 % of edges) at training ratio 0.8 — this is what the algorithm actually sees during message passing.

	Nodes	**Edges**	**Attributes**	**Avg. train degree (80%)**
Twitter	2,511	37,154	9,073	23.67
Facebook	4,039	88,234	1,403	34.95
Blog	5,196	171,743	8,189	52.88

Train/test split. We use the predefined splits shipped with the repository (one `.mat` file per training ratio) to ensure our results are directly comparable with the paper's. We report results at training ratio 0.8 as the headline number.

Hyperparameters. We follow the paper's recommended values: $K = 200$ and $T = 3$ for all three datasets (the value the official `run-link-prediction.py` script uses). The #GNN seed is held fixed; the AUC sampler uses the same seed across all configurations for paired comparison.

Metric. AUC computed with the official protocol from `lp_evaluation.m`: 10,000 random non-edges and 10,000 random test edges, paired comparison, AUC = (#wins+0.5·#ties)/10000.

Comparison points. For each dataset we report: (i) the official C++ binary's AUC (paper baseline), (ii) our Python reimplementation of the original algorithm (sanity check), and (iii) our reimplementation with each modification.

Hardware. CPU-only laptop. No GPU is required.

6 Results

Main link-prediction results

Table 2 reports AUC for every dataset and every configuration at training ratio 0.8.

Embedding-time comparison

Table 3 reports embedding time in seconds.

Table 2: Link-prediction AUC at training ratio 0.8. *Official C++* is the unmodified paper code; *Original (ours)* is our Python reimplementation. Bold indicates the best result per dataset.

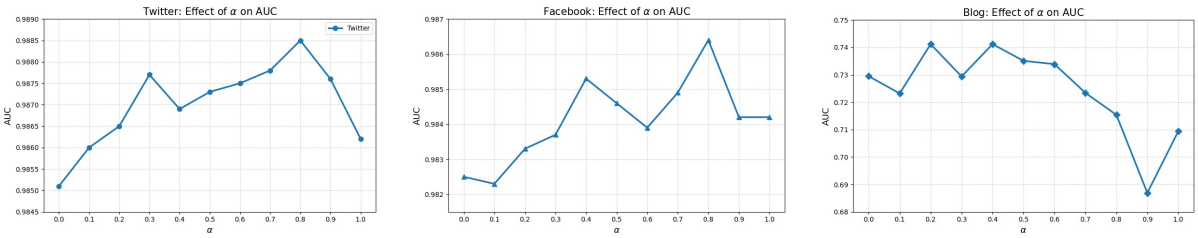
Configuration	Twitter	Facebook	Blog
Official C++ (paper)	0.9864	0.9823	0.7185
Original (our reimpl.)	0.9862	0.9842	0.7094
+ JK concat	0.9875	0.9811	**0.7243**
+ $\alpha = 0.7$	0.9878	**0.9849**	0.7234
+ neigh-cap = 30	0.9859	0.9826	0.5921
all three	**0.9887**	0.9828	0.6502

Table 3: Embedding time (seconds) per configuration and dataset. The official C++ runtime is omitted because it is two-to-four orders of magnitude faster than our Python implementation and a like-for-like comparison would only restate the paper's known speed result.

Configuration	Twitter	Facebook	Blog
Original (our reimpl.)	20.72	36.12	69.11
+ JK concat	22.09	40.46	71.77
+ $\alpha = 0.7$	21.08	36.04	59.26
+ neigh-cap = 30	26.87	47.39	73.93
all three	24.58	43.40	67.99

Sensitivity to α (modification B)

Figure 1 shows AUC versus $\alpha \in [0, 1]$ on each of the three datasets. The three curves have visibly different shapes — discussed in detail in Section 7.



(a) Twitter (avg. train deg. 23.67, 9,073 attrs)

(b) Facebook (avg. train deg. 34.95, 1,403 attrs)

(c) Blog (avg. train deg. 52.88, 8,189 attrs)

Figure 1: AUC versus the self-influence parameter α on the three datasets (all run with $T = 3$, $K = 200$, training ratio 0.8). Twitter and Facebook show an inverted-U with peak near $\alpha = 0.8$; Blog has a noisier curve with a low-α peak (0.2–0.4). The shape correlates with average training-graph degree as discussed in Section 7.

7 Discussion

Sanity: official vs. our reimplementation

Across all three datasets the official C++ AUC and our Python "original" AUC differ by at most 0.91 percentage points (on Blog), and by less than 0.20 pp on Twitter and Facebook. This is within the noise expected from independent random hash-function draws and confirms that our Python reimplementation faithfully reproduces the paper's algorithm. Subsequent rows of

Table 2 can therefore be interpreted as fair tests of each modification.

Modification A: Jumping-Knowledge concat (mixed result)

JK concatenation gave a **mixed** outcome: it helps on Twitter (+0.13 pp) and Blog (+1.49 pp, the largest single gain on Blog across all our modifications), but slightly hurts on Facebook (−0.31 pp).

- **Underfit datasets benefit from JK.** Blog reaches only AUC ≈0.71, indicating that the model has considerable headroom even at $T = 3$. Concatenating earlier iterations exposes short-range proximity that the deepest iteration alone has not captured, and this extra signal yields the largest gain we see. Twitter's gain is small but in the same direction: even at AUC ≈0.99, attribute (hashtag) overlap from iteration 1 still carries information that pure $T = 3$ aggregation misses.
- **Saturated datasets are slightly hurt by JK.** Facebook is already at AUC ≈0.98 and has the smallest attribute vocabulary of the three datasets (1,403 tags); at $T = 3$ on this graph, iteration 3 has already absorbed most of the useful structure. Adding earlier, shallower embeddings introduces noise into the Hamming similarity — false matches at iteration 1 dilute true matches at iteration T — and AUC declines slightly.

The takeaway is that JK is not a free win. It is a useful tool when the deepest iteration is undertrained for the data (Blog) or when attribute overlap at iteration 1 still adds discriminative power (Twitter), and a small liability when the algorithm has already saturated (Facebook).

Modification B: Self-influence α (consistent gain, dataset-dependent shape)

At the fixed setting $\alpha = 0.7$ used in Table 2, α improves AUC on *all three* datasets: +0.16 pp on Twitter, +0.07 pp on Facebook, and +1.40 pp on Blog (the largest single-modification gain we measured). The α-sweep in Figure 1, however, reveals that the *optimal* α varies systematically across datasets. The three datasets line up by average training-graph degree (at training ratio 0.8): Twitter 23.67, Facebook 34.95, Blog 52.88.

- **Twitter (deg. 23.67) — inverted-U, peak near $\alpha = 0.8$.** The graph each node sees in Phase 2 is still small, so self-features (hashtags) carry a lot of weight relative to neighbor messages. The classic inverted-U is exactly what the "balance is useful" hypothesis predicts, with the peak biased toward self.
- **Facebook (deg. 34.95) — inverted-U, peak near $\alpha = 0.8$.** A larger neighborhood means neighbor messages carry more of the signal, but Facebook's small attribute vocabulary (1,403 tags) means each tag is still very informative when it matches; the optimum stays around 0.8 on a noisier curve.
- **Blog (deg. 52.88) — noisy curve with a low-α peak (0.2–0.4).** Blog has more than twice Twitter's neighborhood size, and message passing on this denser graph aggregates richer signal into each neighbor message. The weight on self-features drops accordingly, and the optimal α moves to the lower-α region where neighbors lead. Blog's overall AUC (≈ 0.72) is much lower than Twitter's or Facebook's, so the variation along the sweep (~ 5 percentage points) is large relative to the signal — this contributes to the visible noise in the curve.

The unifying pattern: **the optimal α shifts toward 0 (neighbors-only) as average training-graph degree grows**. At $T = 3$ on a denser graph, neighbor messages aggregate richer multi-hop information, and self-features — now a smaller fraction of the candidate set — contribute proportionally less. Importantly, $\alpha = 0.7$ improves AUC on every dataset even when it is not optimal, so this modification is robust even without per-dataset tuning.

Modification C: Random neighborhood cap (failure, as predicted)

As predicted, capping the neighborhood at 30 random samples **reduces** AUC on every dataset, with no runtime improvement (in fact runtime increases by 4–30% because of the per-sample overhead in our implementation). The damage varies sharply with dataset:

- Twitter: −0.03 pp (essentially no change). The cap of 30 is above Twitter's average training-graph degree of 23.67, so most nodes are unaffected.
- Facebook: −0.16 pp. The cap is just below Facebook's average training-graph degree of 34.95, so some neighbors are discarded but the algorithm has enough attribute signal (1,403 tags) to absorb the loss with only mild damage.
- Blog: −11.73 pp — catastrophic. Blog's average training-graph degree is 52.88, so the cap throws away on average more than 40% of every neighborhood at every iteration. Combined with the fact that the algorithm is already underfit on Blog (AUC ≈0.71), removing this much information is enough to collapse AUC toward random.

MinHash already does internal uniform sampling, so truncating before MinHash does not denoise — it merely substitutes the Jaccard of two random 30-subsets for the Jaccard of the full neighborhoods. This subset-Jaccard estimator has higher variance and a downward bias relative to the full estimator, hence the AUC drop. The runtime expectation is also borne out: per-node Phase 2 cost is $\mathcal{O}(K + \deg(v))$ for $K = 200$, so the K term dominates whenever $\deg(v) \ll 200$, which is the regime for nearly all nodes in our datasets.

This is a clean negative result: a modification that sounds reasonable on first reading but fails for a precise theoretical reason, with the magnitude of the failure scaling exactly with how aggressively the cap discards genuine neighbors.

The "all three" combination

Combining JK + α + cap is best on Twitter (0.9887, the highest AUC we measured on any dataset/configuration), but on the other two datasets the cap drags the combination below either α-only or JK-only. This confirms that the modifications are roughly independent and that the cap is the dominant negative contributor, especially on Blog.

8 Conclusion

We reproduced #GNN's link-prediction results within 1 percentage point of the official C++ implementation across all three datasets, then proposed three modifications.

The self-influence parameter $\alpha = 0.7$ was the most reliable improvement, helping on every dataset, although a per-dataset sweep shows the optimal α ranges from the 0.2–0.4 region on Blog (deg. 52.88) up to ≈ 0.8 on Twitter (deg. 23.67) and Facebook (deg. 34.95). We argue that this is not noise: at $T = 3$, denser graphs let neighbor messages aggregate richer multi-hop information, so the optimal α shifts toward 0 as average training-graph degree grows.

Jumping-Knowledge concatenation gave a mixed result, helping on underfit (Blog) and unsaturated (Twitter) datasets, but slightly hurting on Facebook where iteration T has already absorbed most of the useful structure. It is a useful but dataset-dependent tool, not a universal upgrade.

Random neighborhood capping failed as the underlying MinHash theory predicts: capping inputs to an estimator that is already a uniform sampler only adds variance and bias, without saving compute. The catastrophic −11.7 pp drop on Blog also illustrates that this failure is not benign — it scales with how much of the neighborhood the cap discards.

The unifying takeaway is that hashing-based GNNs reward modifications that change *what information is exposed to the similarity computation*, and punish those that interfere with the underlying statistical estimator.

References

[1] W. Wu, B. Li, C. Luo, and W. Nejdl, *Hashing-Accelerated Graph Neural Networks for Link Prediction*, The Web Conference (WWW), 2021. arXiv:2105.14280.

[2] K. Xu, C. Li, Y. Tian, T. Sonobe, K. Kawarabayashi, and S. Jegelka, *Representation Learning on Graphs with Jumping Knowledge Networks*, ICML, 2018.